如何在大型应用架构中设计微前端方案

By 淘系前端开发 - 夏温武（鲲尘）

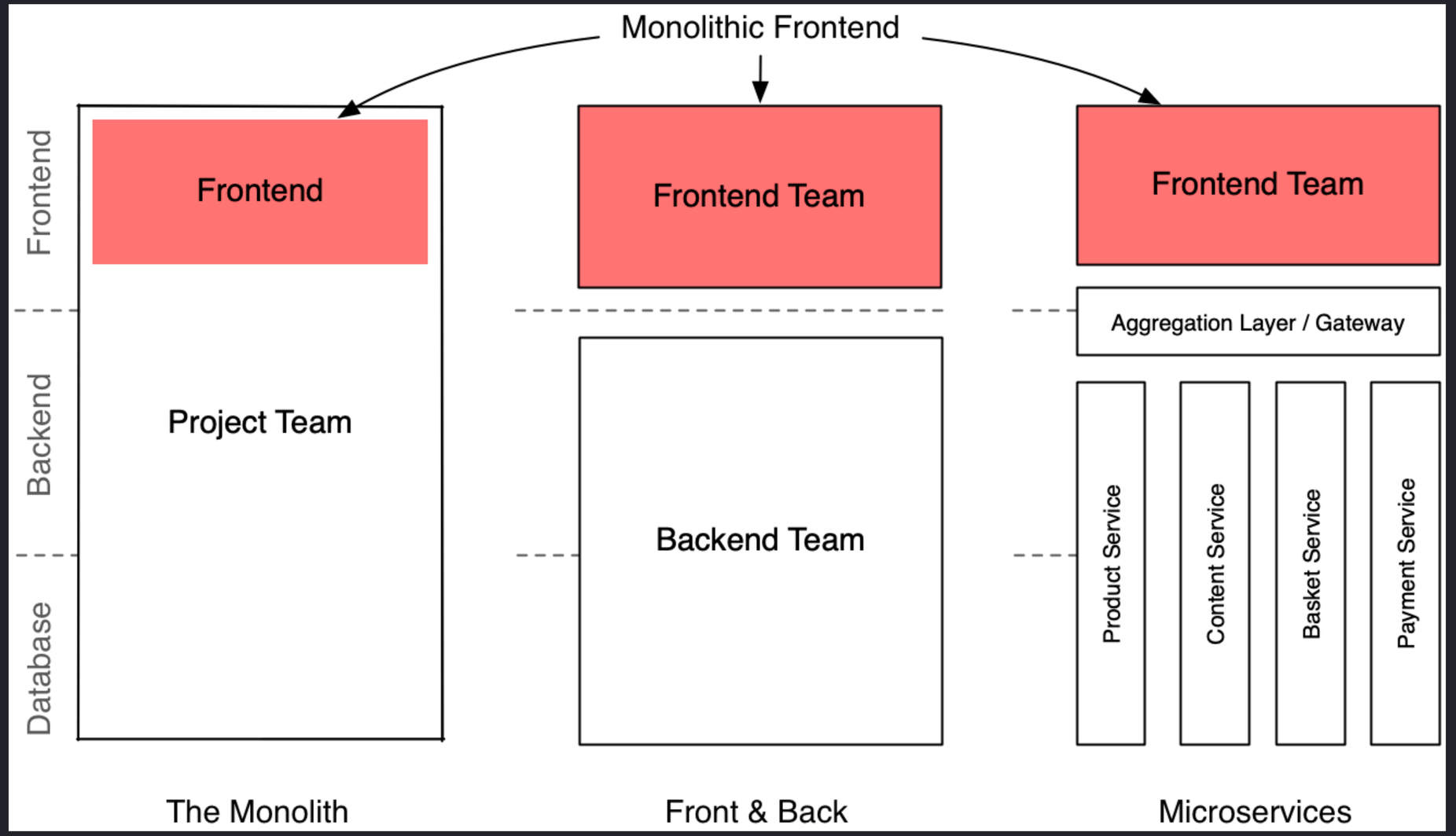
01. 概念介绍

02. 业务背景

03. 架构设计

04. 业务价值

微前端 – 概念介绍



微前端是一种类似微服务的架构，它将微服务的概念应用于浏览器端

02. 业务背景

团队平台系统多且相互独立

平台系统体量大且页面数量多



建立体验良好、可持续维护的系统

微前端 - 业务背景 - 技术选型

	巨石应用	iframe	微前端	框架组件
一个系统	√	√	√	x
用户体验	√	x	√	x
技术复杂度	√	x	x	x
系统健壮性	x	√	√	√
开发体验	x	√	√	√
二方接入	-	√	√	√
三方接入	x	√	x	√
子应用部署成本	-	x	√	x

- **巨石应用**：随着页面增多，开发效率与稳定性无法满足要求 🤕
- **`iframe`**：除了用户体验问题，其实是个很不错的方案 💀
- **框架组件**：不满足一个系统的心智，同时长期维护成本高 💩
- **微前端**：在大型系统的业务场景下，体验和效率的平衡点 🚀

微前端 – 业务背景 – 微前端方案

入驻

账号管理

内容管理

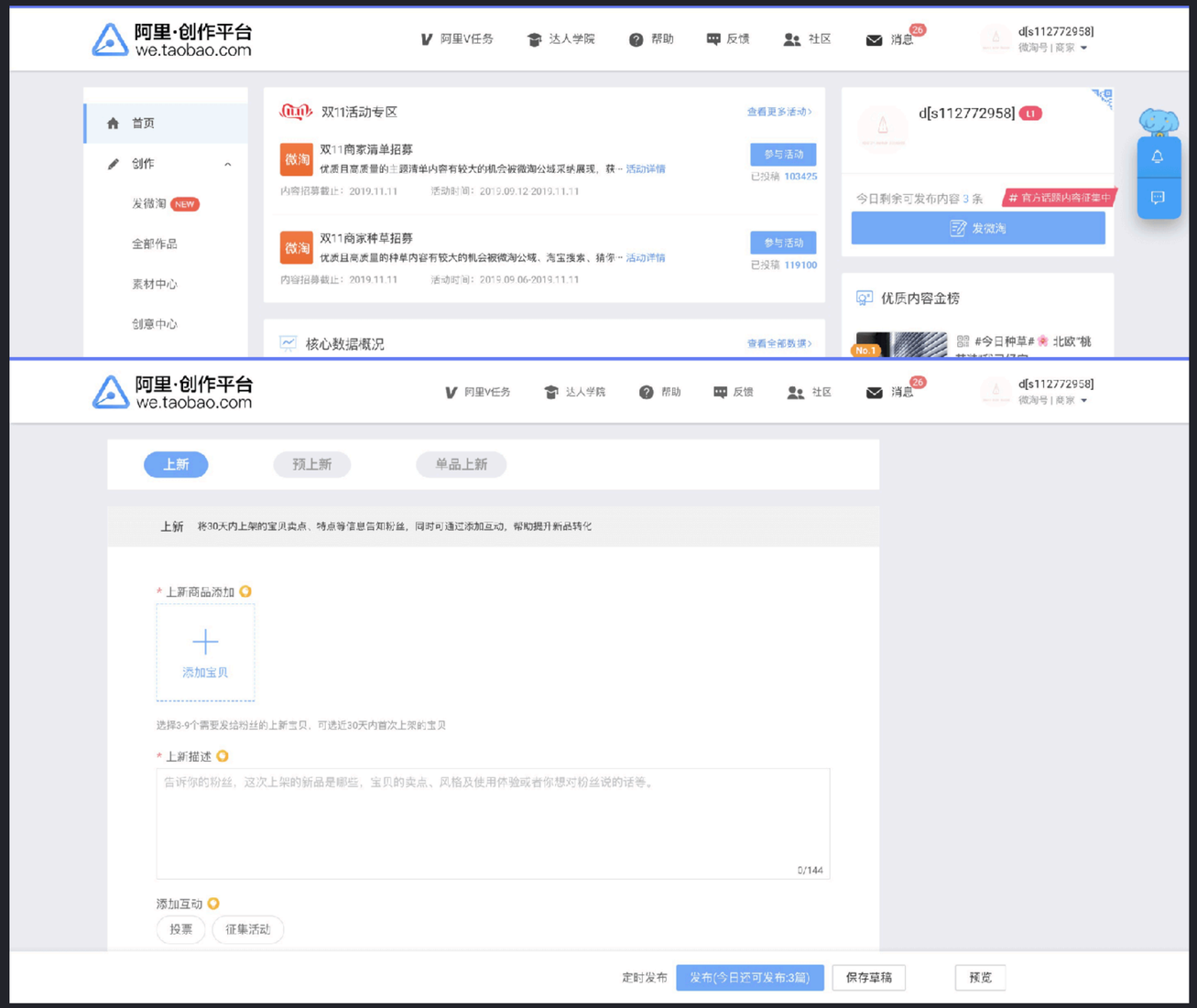
内容发布

粉丝运营

数据分析

...

50+
页面



阿里创作平台

03. icestark 架构设计

01.微前端方案

02.架构设计理念

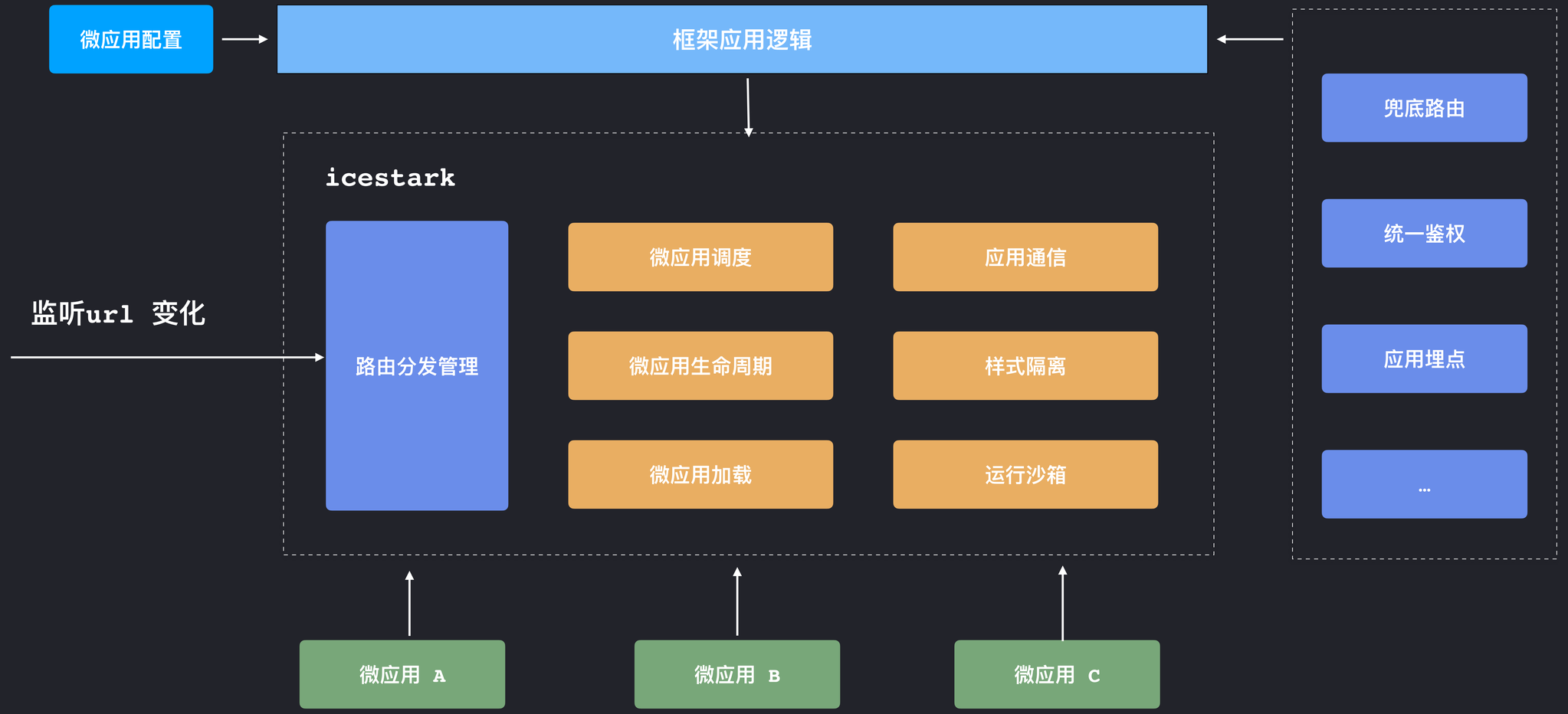
03.核心概念

04.工作原理

05.隔离

06.微模块

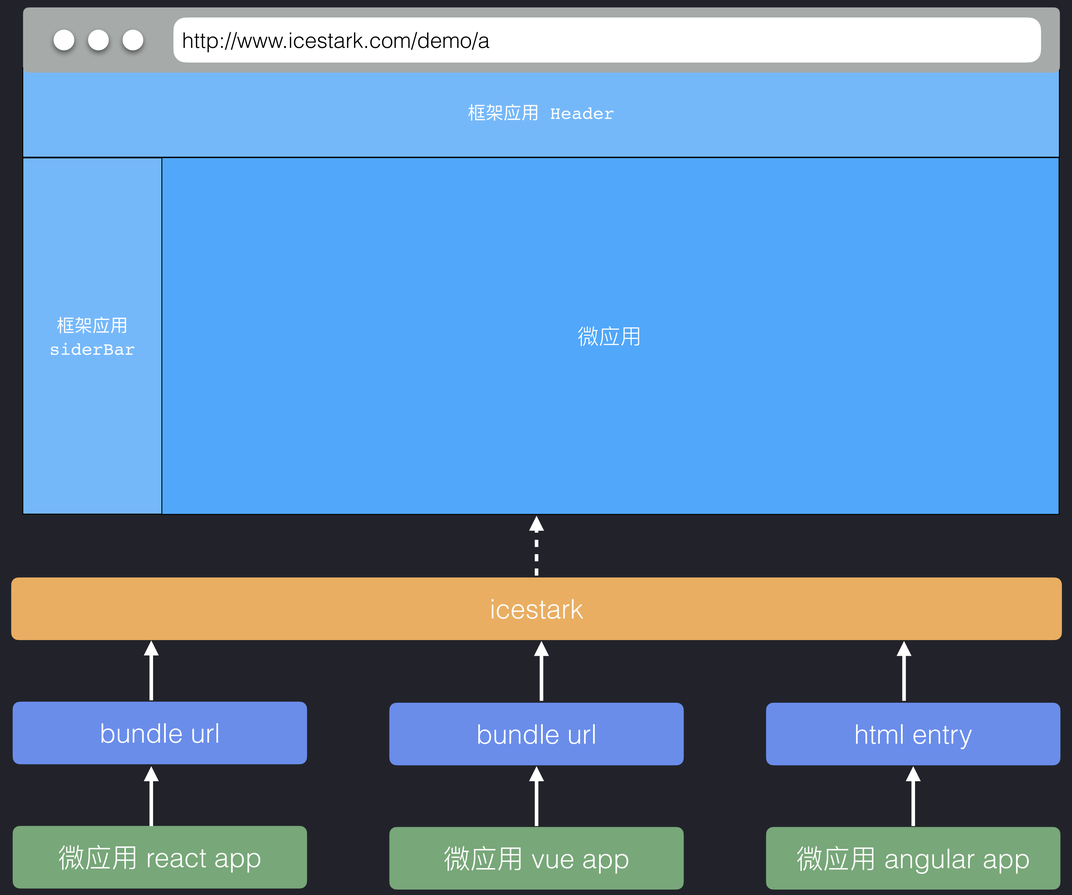
微前端 - 微前端方案


微应用配置
框架应用逻辑
兜底路由
统一鉴权
应用埋点
...
icestark
监听url 变化
路由分发管理
微应用调度
应用通信
微应用生命周期
样式隔离
微应用加载
运行沙箱
微应用 A
微应用 B
微应用 C

微前端 - 架构设计理念


单一体系技术栈统一
技术栈无关
开发体验一致
不用学习新概念新流程
中心化路由
独立开发部署
路由管理
应用分发
独立开发
独立部署

微前端 - 核心概念



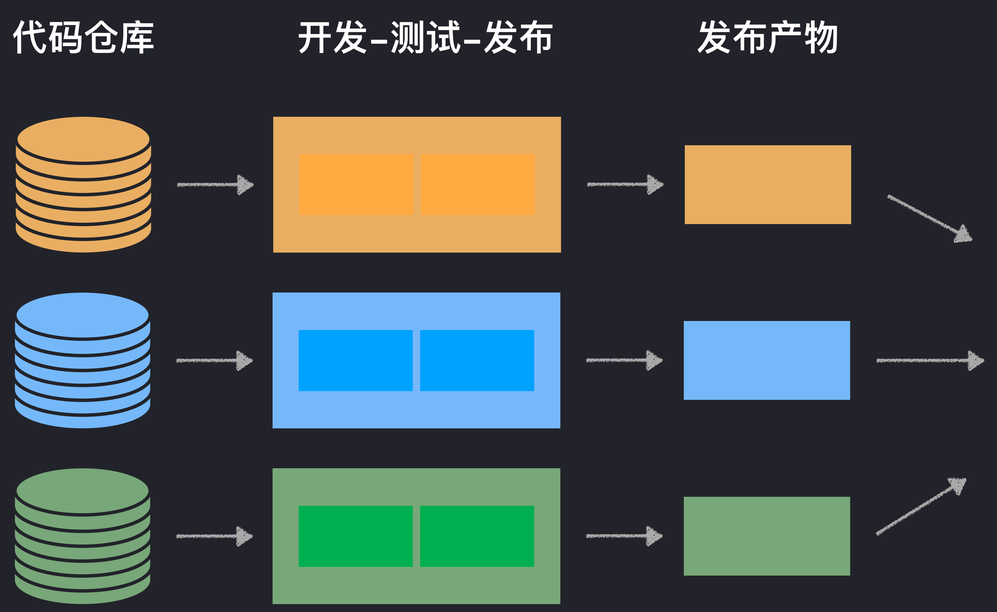
- **框架应用**：负责 Layout 以及微应用配置与注册渲染

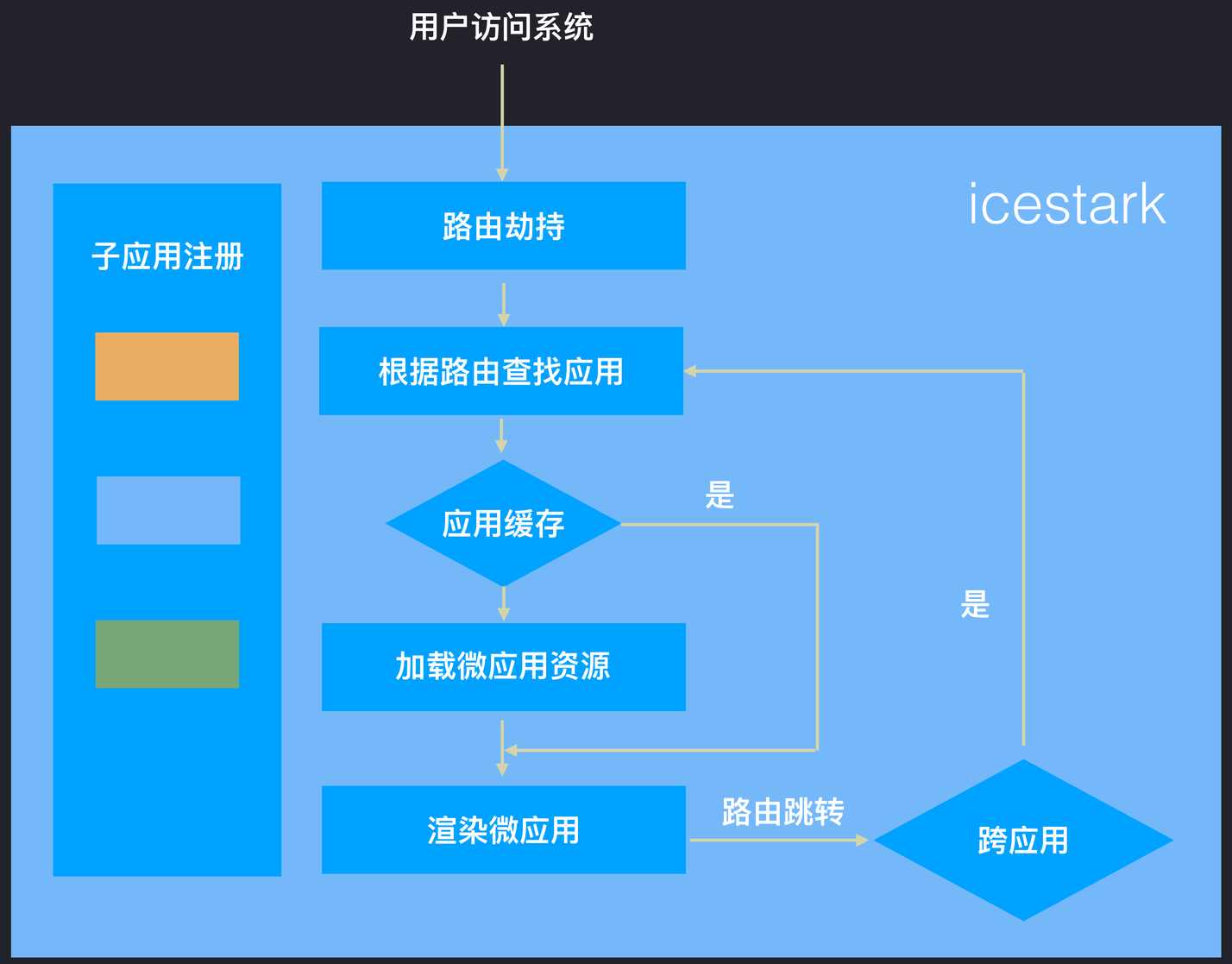
- **微应用**：按业务功能纬度拆分应用，通常是一个 SPA 应用，至少包含一个页面/路由

微前端 - 工作原理 - 微应用注册

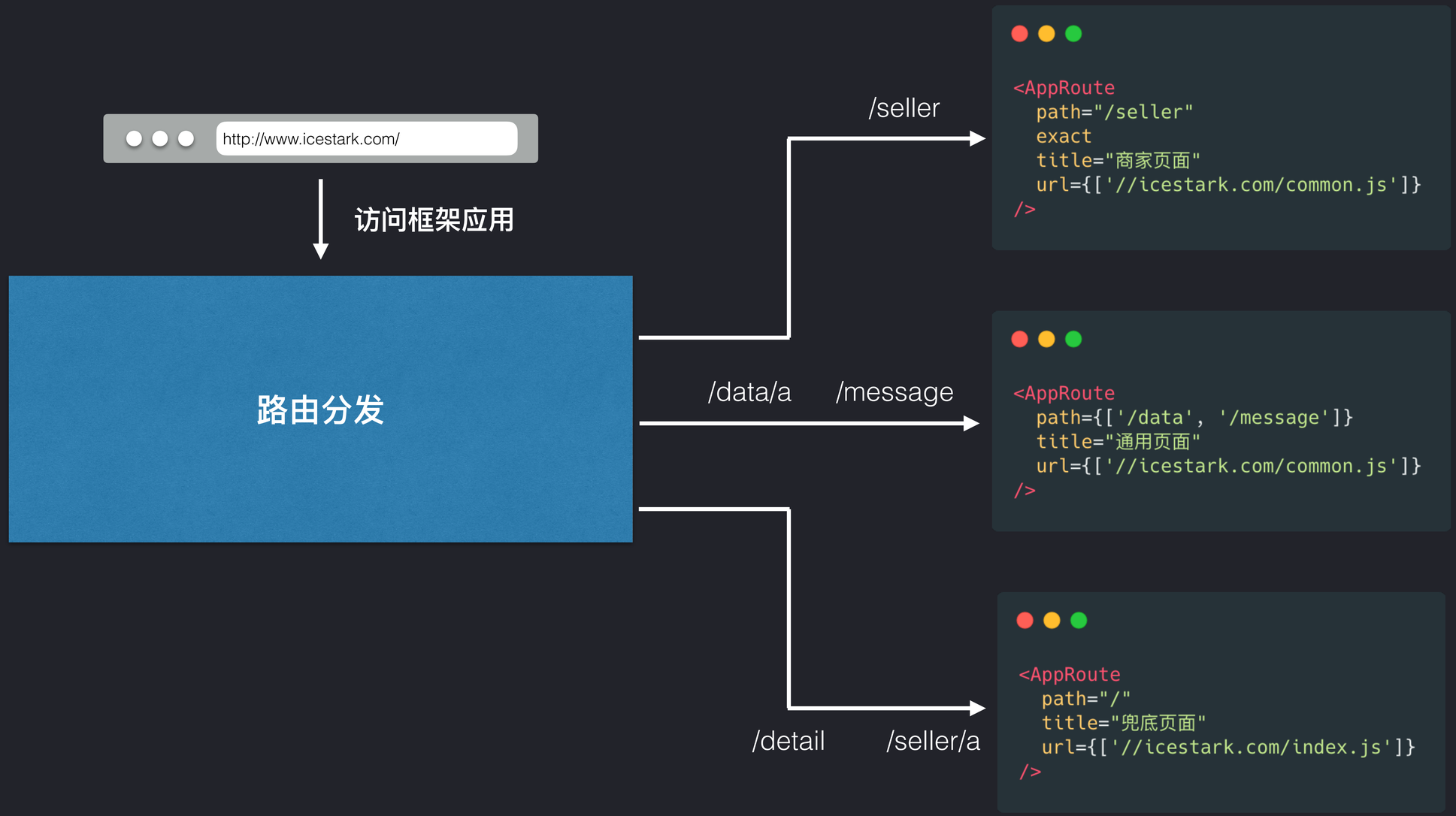
```
<AppRouter>
  <AppRoute
    path={['/', '/message', '/about']}
    exact
    title="通用页面"
    url={['//unpkg.com/icestark-child-common/build/js/index.js']}
  />
  <AppRoute
    path="/seller"
    title="商家平台"
    url={[
      '//unpkg.com/icestark-child-seller/build/js/index.js',
      '//unpkg.com/icestark-child-seller/build/css/index.css',
    ]}
  />
  <AppRoute
    path="/customer"
    title="用户平台"
    entry="https://ice.alicdn.com/icestark/child-common-angular/index.html"
  />
</AppRouter>
```

微前端 - 工作原理 - 流程

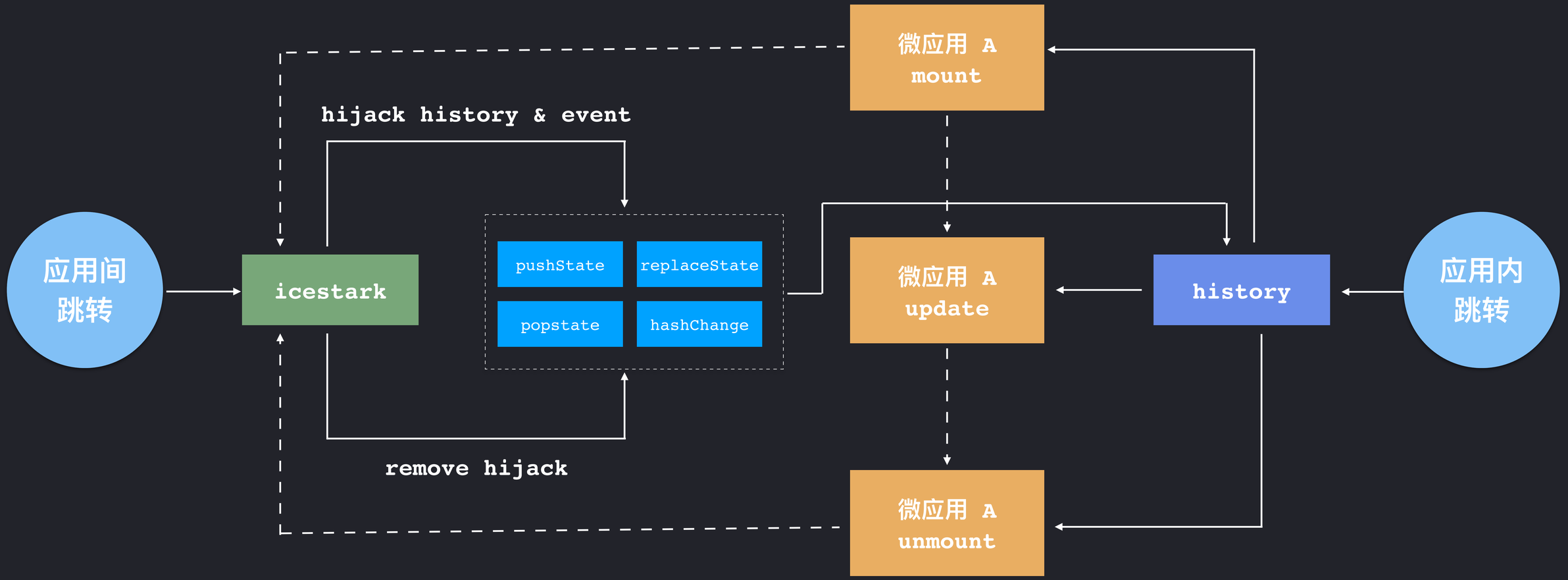

代码仓库
开发-测试-发布
发布产物


用户访问系统
icestark
子应用注册
路由劫持
根据路由查找应用
应用缓存
是
加载微应用资源
渲染微应用
路由跳转
跨应用
是

微前端 - 工作原理 - 路由规则



微前端 - 工作原理 - 路由劫持



微前端 - 微应用加载渲染

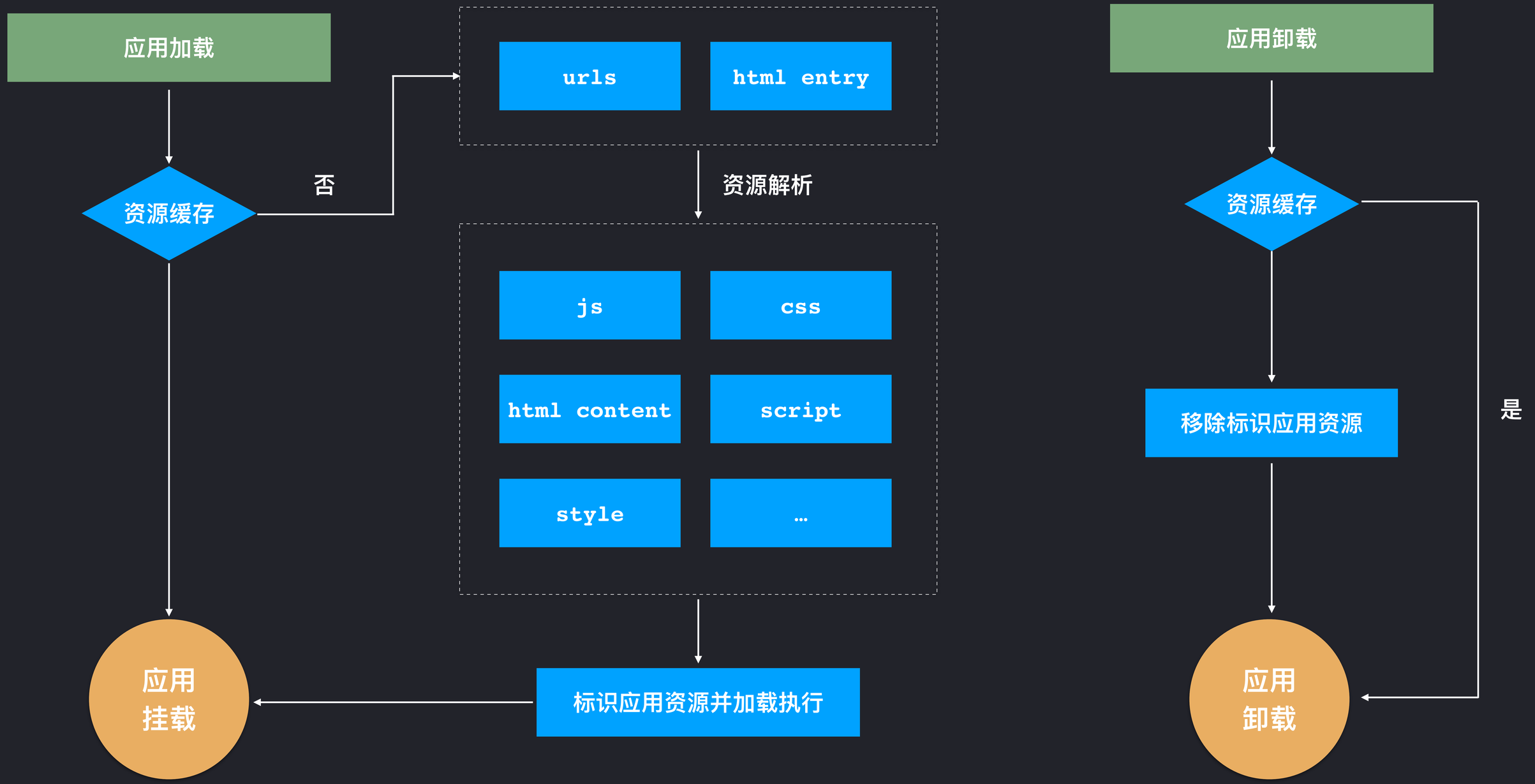
```
import ReactDOM from 'react-dom';
import { isInIcestark, getMountNode, registerAppEnter, registerAppLeave } from '@ice/stark-app';
import router from './router';

if (isInIcestark()) {
  const mountNode = getMountNode();
  registerAppEnter(() => {
    ReactDOM.render(router(), mountNode);
  });
  registerAppLeave(() => {
    ReactDOM.unmountComponentAtNode(mountNode);
  });
} else {
  ReactDOM.render(router(), document.getElementById('ice-container'));
}
```

registerAppEnter：注册应用挂载生命周期

registerAppLeave：注册应用卸载生命周期

微前端 - 微应用加载渲染


应用加载
资源缓存
否
urls
html entry
资源解析
js
css
html content
script
style
…
标识应用资源并加载执行
应用
挂载
应用卸载
资源缓存
是
移除标识应用资源
应用
卸载

应用间数据通信

```
// 微应用
import { store } from '@ice/stark-data';

// 监听语言变化
store.on('language', (lang) => {
  console.log(`current language is ${lang}`);
}, true);

// 获取当前用户
const userInfo = store.get('user');
```

```
// 框架应用
import { store } from '@ice/stark-data';

const userInfo = { name: 'Tom', age: 18 };
store.set('language', 'CH'); // 设置语言
store.set('user', userInfo); // 设置登录后当前用户信息

setTimeout(() => {
  store.set('language', 'EN');
}, 3000);
```

应用间事件通信

```
// 框架应用
import { event } from '@ice/stark-data';

event.on('freshMessage', () => {
  // 重新获取消息数
});
```

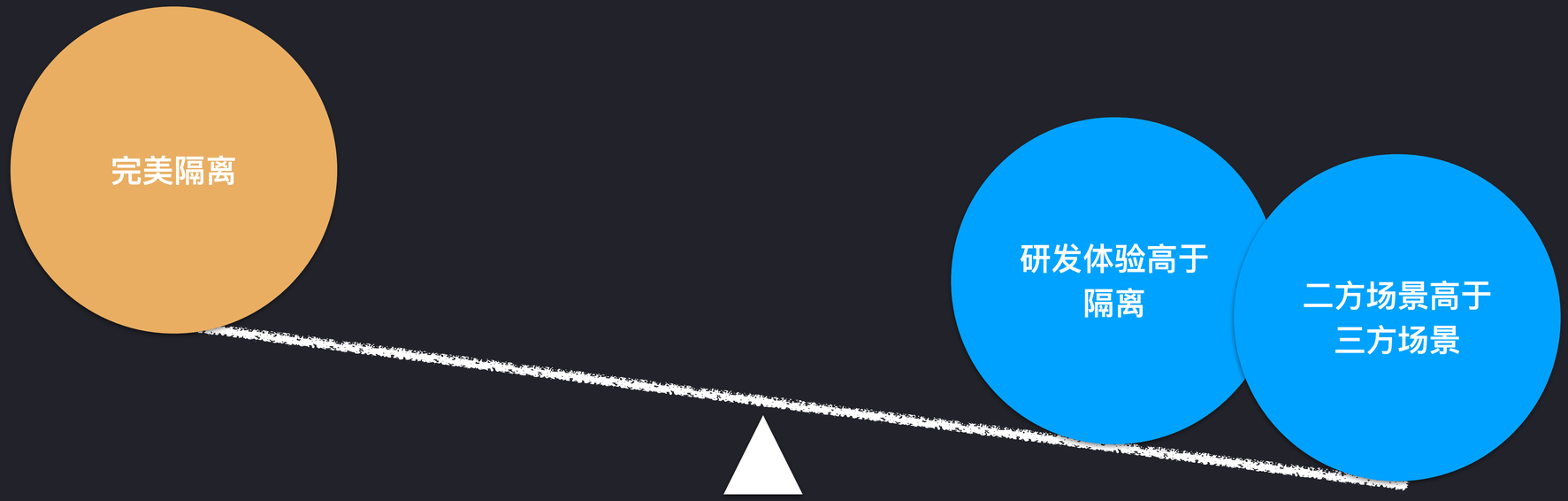
```
// 微应用
import { event } from '@ice/stark-data';

event.emit('freshMessage');
```

微前端 - 应用通信


框架应用
微应用 A
微应用 B
微应用 C
window
@ice/stark-data
emit
on event
off event
has event

微前端 - 隔离 - 原则



微前端 - 隔离 - 样式

业务代码样式隔离

CSS Modules

```css
// CSS 样式
.test {
  color: #fff;
}

// CSS Modules 样式
.Homr--test-3XuHNoA {
  color: #fff;
}
```

基础组件样式隔离

基础组件 namespace

```css
// 微应用基础样式
.next-btn {
  color: #fff;
}

// 框架应用 prefix 修改
.next-icestark-btn {
  color: #fff;
}
```

shadow DOM?

- 处理逃逸容器节点
- 事件机制兼容
- . . .

微前端 - 隔离 - 脚本


code
创建沙箱
const sandbox = new Proxy(Object.create(null), {
get(){},
set(){},
has(){},
...
});
执行代码
const execScript = `with (sandbox) {;${script}\n}`;
const execCode = new Function('sandbox', execScript);
execCode(sandbox);
window 快照
记录与恢复
window

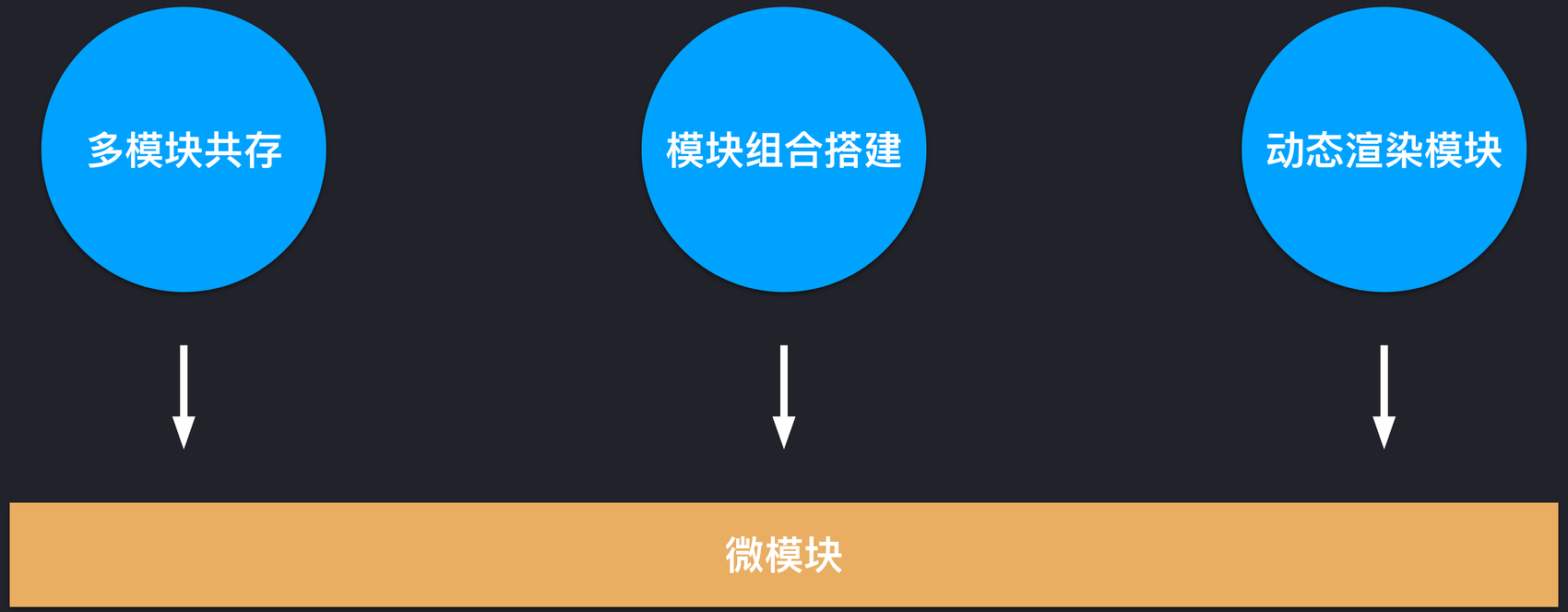
三方接入隔离方案

iframe

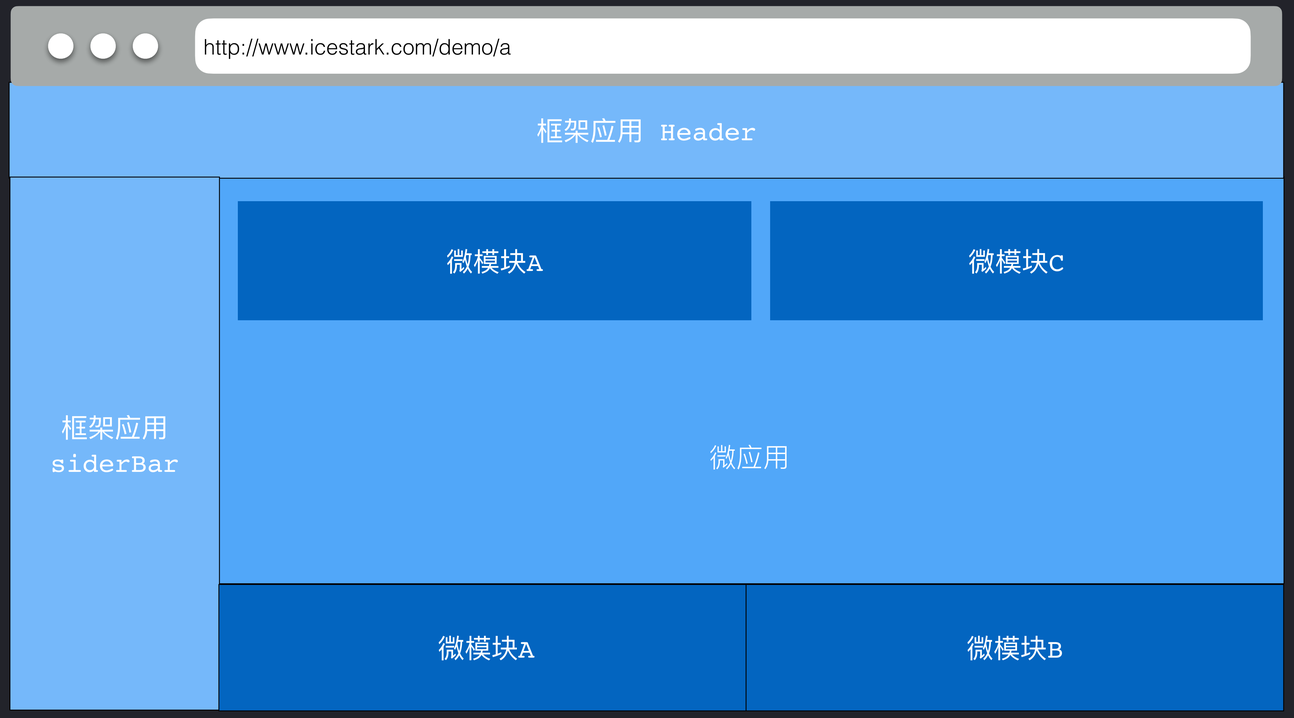
```
import React from 'react';
import { AppRouter, AppRoute } from '@ice/stark';
import BasicLayout from '@/layouts/BasicLayout';

const App = () => {
  render() {
    return (
      <BasicLayout>
        <AppRouter>
          <AppRoute
            path="/seller"
            basename="/seller"
            title="通用页面"
            render={() => {
              return <iframe src="http://pathtoiframe" />
            }}
          />
        </AppRouter>
      </BasicLayout>
    );
  }
}
```

微前端 – 微模块 – 业务场景



微前端 - 微模块



- **微模块**：多模块共存/不耦合路由/独立完成模块功能

- 微模块标准：在标准 UMD 模块规范基础上，定义 mount/unmount 生命周期

04. 业务价值

现状和痛点

- 项目量级变大
- 开发效率低
- 技术栈迁移成本高
- 二方业务接入成本高

微前端





微前端 - 业务价值 - portal 系统

现状和痛点

- 多系统独立无管控
- 重复建设严重
- 操作效率低
- 体验不一致

微前端



- 可管理性
- 开发部署成本
- 操作一致性

微前端 – 业务价值 – 业务落地

阿里设计服务平台
Alibaba Design

CONE

YOUKU
统一运营平台

阿里·创作平台
we.taobao.com

运营工作台

同心

商家中心

财鲸 · 应收管理平台

伏羲 | 智能制造平台

20+ 平台级应用

微前端 - 总结

业务背景

大型应用 + portal 系统面临的技术挑战

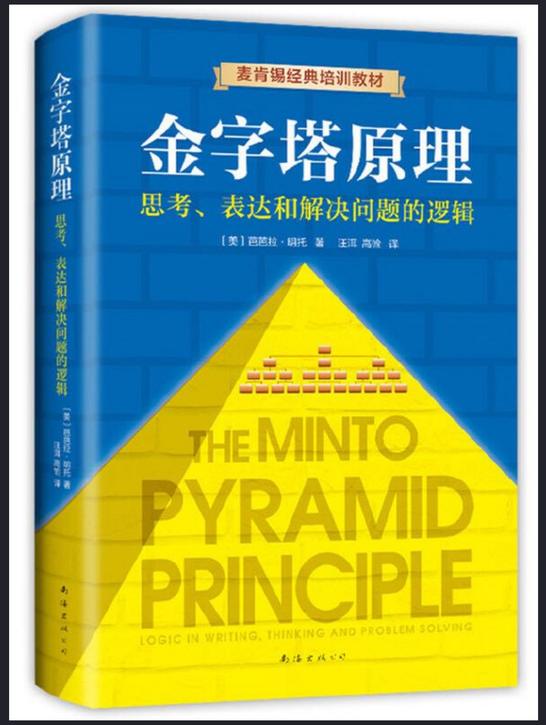
架构设计

框架应用 + 微应用的微前端解决方案

业务价值

基于微前端架构建立可持续维护的系统

微前端 - 书籍推荐



《金字塔原理》

培养自己说/想/做/表演的能力

- 表达逻辑
- 思考逻辑
- 解决问题逻辑
- 演示的逻辑

THANKS & QA

👏欢迎关注 icestark：https://github.com/ice-lab/icestark



